

DEEP-PIPELINE

Geliştirme ve Süreç Raporu
Yazılım Mühendisliği Dokümantasyonu

TEKNOFEST 2026 — E-Ticaret Hackathonu

Tarih: 18.06.2026

Takım: Deep-Pipeline

1. Giriş ve Amaç

Bu rapor, Deep-Pipeline projesinin yazılım geliştirme sürecini, alınan mühendislik kararlarını ve uygulanan refactoring işlemlerini detaylı şekilde dokümente etmektedir. Raporun amacı, projenin başlangıç noktasından yarışmaya hazır hale gelişine kadarki tüm adımları şeffaf bir şekilde sunmaktır.

2. Başlangıç Durumu (Refactoring Öncesi)

Proje başlangıcında, sistem iki büyük monolitik Python dosyasından oluşuyordu: `teknofest-benchmark-runner.py` (1002 satır) ve `teknofest-experiment-tracker.py`. Bu dosyalar, kök dizinde dağınık şekilde bulunuyordu ve modüler bir yapıya sahip değildi.

Tespit edilen sorunlar:

- Konfigürasyon değerleri kodun içine gömülü (hardcoded)
- Test, eğitim ve değerlendirme kodları iç içe geçmiş
- Yeniden kullanılabilirlik (reusability) çok düşük
- `requirements.txt`, `Dockerfile` ve `.gitignore` eksik
- Deney takibi (experiment tracking) yapılandırılmamış

3. Yazılım Refactoring Süreci

3.1 Dosya Yapısı Dönüşümü

Dağınık haldeki tüm script dosyaları, endüstriyel standartlara uygun bir src/ dizin yapısına taşındı. Hedeflenen mimari şöyledir:

ÖNCEKİ YAPI:

```
teknofest-benchmark-runner.py (1002 satir, monolitik)
teknofest-experiment-tracker.py (monolitik)
src/ (bos alt dizinler)
```

SONRAKİ YAPI:

```
configs/base_config.yaml
configs/model/distilberturk.yaml
src/data/preprocessor.py
src/features/brand_features.py
src/models/cross_encoder.py
src/training/distillation.py
src/experiment/config_loader.py
src/experiment/tracker.py
scripts/run_experiment.py
scripts/run_benchmark.py
scripts/run_dashboard.py
```

3.2 Konfigürasyon Sistemi

Tüm deney parametreleri (model tipi, öğrenme oranı, batch boyutu, distillation alfa değeri, threshold) YAML dosyalarına çıkarıldı. Bu sayede bir deney başlatmak için sadece 'python scripts/run_experiment.py --config configs/base_config.yaml' komutunu çalıştırmak yeterli hale geldi.

3.3 MLOps Entegrasyonu

MLflow tabanlı deney takip sistemi kuruldu. Her deney otomatik olarak parametreleri, metrikleri ve model ağırlıklarını kaydeder. Bu sayede yüzlerce deney arasında kolayca karşılaştırma yapmak mümkündür.

4. Metin Ön İşleme Modülü

Türkçe e-ticaret metinleri için özel bir ön işleme modülü geliştirildi (src/data/preprocessor.py). Bu modül şunları yapar:

- Türkçe Özel Küçük Harf: Python'ın varsayılan lower() fonksiyonu Türkçe İ/i ve I/ı karakterlerini doğru dönüştüremez. Özel bir _turkish_lower() fonksiyonu yazıldı.
- Yazım Hatası Düzeltme: SymSpell kütüphanesi ile edit distance tabanlı hızlı yazım düzeltme. E-ticaret özel sözlük desteği.
- Lemmatizasyon: Zeyrek (Zemberek Python portu) ile morfolojik analiz. 'ayakkabılar' → 'ayakkabı' gibi dönüşümler.

5. Özellik Çıkarımı (Feature Engineering)

Modelin başarısı, ham metin dışında ek özellikler kullanılmasına bağlıdır. Oluşturulan özellikler şunlardır:

- Fuzzy Brand Match Score: RapidFuzz kütüphanesi ile sorgu ve marka arasındaki karakter benzerlik oranı (0-100 float).
- Has Brand Match: Fuzzy skorun 85 eşliğini aşip aşmadığını gösteren binary (0/1) gösterge.

6. Model Eğitim Stratejisi

6.1 Cross-Encoder

DistilBERTTurk tabanlı Cross-Encoder modeli, sorgu-ürün çiftlerini birleştirerek 3 sınıflı (Alakasız / Kısmen Alakalı / Çok Alakalı) sınıflandırma yapar. Batch boyutu 32, max_length 128 token olarak ayarlanmıştır.

6.2 Teacher-Student Distillation

Büyük BGE-M3 modeli (Teacher) tarafından üretilen soft-label'lar, DistilBERTTurk (Student) modeline Margin-MSE loss fonksiyonu ile aktarılır. Margin-MSE, klasik MSE'den farklı olarak pairwise ranking sinyali taşır: iki ürün arasındaki skor farkının (margin) Teacher ve Student modellerinde tutarlı olmasını hedefler.

$$\text{Loss} = \alpha \times \text{CE_Loss} + (1 - \alpha) \times \text{MarginMSE_Loss}$$

$\alpha = 0.7$ (base_config.yaml'dan okunur)

7. Web Arayüzü Geliştirme

Projenin sunumu için Next.js (App Router) tabanlı, Brutalist ve Fütüristik bir web arayüzü geliştirildi. Kullanılan teknolojiler:

- Next.js 15 — React Server Components
- Three.js / @react-three/fiber — 3D veri simülasyonu
- Framer Motion — Mikro animasyonlar
- GSAP — Scroll-triggered parallax efektleri
- Zustand — Teknik/Basit mod state yönetimi

Arayüz, kullanıcıyı bir 'terminal boot simülasyonu' ile karşılar, ardından e-ticaretin veri kaosunu 3D parçacık animasyonu ile görselleştirerek, çözümün nasıl çalıştığını interaktif modüllerde anlatır. Sağ üstteki 'Teknik/Basit' toggle ile tüm içerik anında değişir.

8. Dağıtım ve DevOps

Proje, Docker tabanlı offline dağıtım için yapılandırılmıştır. Dockerfile, tüm bağımlılıkları ve model ağırlıklarını içerir. FastAPI üzerinden REST API sunulur.

```
# Docker ile calistirma
docker build -t deep-pipeline .
docker run -p 8000:8000 \
  -v ./local_model:/app/local_model \
  deep-pipeline
```

9. Sonuç ve Gelecek Adımlar

Deep-Pipeline, dağınık bir prototipten profesyonel bir MLOps pipeline'ına başarıyla dönüştürülmüştür. Konfigürasyon odaklı mimari sayesinde hızlı deney döngüsü, Docker ile offline dağıtım, ve XAI dashboard ile şeffaf karar verme mekanizması sağlanmıştır.

Gelecek adımlar:

- Yarışma veri seti açıklandığında baseline eğitimi
- Multi-Seed Ensemble optimizasyonu
- ONNX Runtime ile CPU inference optimizasyonu
- Streamlit dashboard'ın gerçek verilerle test edilmesi